Дефиниция 1 *Нека $G = (V, E)$ е неориентиран граф. Всяко дърво D от вида (V, E'), където $E' \subseteq E$ наричаме* покриващо дърво *на графа G.*

Теорема 2 *Графът G притежава покриващо дърво тогава и само тогава, когато G е свързан.*

Доказателство. ($\Longrightarrow$) Нека D е покриващо дърво на G. Тъй като D е свързан граф, от всеки връх има път в D до всеки друг връх. Но всеки път в D е също път в G. Така G също е свързан граф.

($\Longleftarrow$) Нека G е свързан граф. Изпълняваме следната процедура:

$H = G$; докато в H има цикъл, премахваме ребро от цикъла;

Ясно е, че процедурата приключва, тъй като ребрата на G са краен брой. Резултатът от процедурата очевидно е ацикличен граф. Остава да проверим, че е и свързан. За целта ще докажем, че след всяка итерация на цикъла H остава свързан граф. Нека $H = (V, E')$ е свързан и реброто $e \in E'$ участва в цикъл, $e = \{x, y\}$, като цикълът има вида $x, y, x_1, \ldots, x_k, x$ за $k \geq 1$. Да вземем два произволни върха $v, v' \in V$, $v \neq v'$. Има път от v до v' в H. В този път можем да премахнем всяко участие на реброто e по следния начин: заменяме x, y с $x, x_k, \ldots, x_1, y$ и също заменяме y, x с $y, x_1, \ldots, x_k, x$. Получаваме отново път от v до v', но в него не участва реброто e. Следователно, $(V, E' \setminus \{e\})$ също е свързан граф.

Забележете, че ако G не е свързан, процедурата произвежда по едно покриващо дърво за всяка свързана компонента на G, което можем да наречем гора от покриващи дървета.

Обхождане на граф в ширина

Даден е неориентиран граф $G = (V, E)$. Целта е систематично да обходим неговите върхове, като се движим само по ребрата.

При обхождането в ширина разбиваме V на нива $L_0, L_1, \ldots, L_k$, като използваме следната процедура:

1. Избираме начален връх $r \in V$, $L_0 = \{r\}$, $l = 0$.

2. Ако $L_0 \cup L_1 \cup \ldots \cup L_l = V$, $k = l$ и процедурата приключва (всички върхове са обходени).

3. В противен случай, образуваме ново ниво

$$L_{l+1} \;=\; \{v \in V \mid v \notin L_0 \cup L_1 \cup \ldots \cup L_l \;\&\; \exists u \in L_l \; \{u, v\} \in E\}.$$

Повишаваме l с 1 и се връщаме на 2.

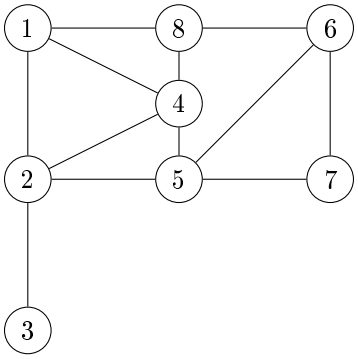
По друг начин можем да кажем, че в L_{l+1} включваме всички необходени (неучастващи в предходните нива) съседи на върховете от L_l.

На базата на алгоритъма за обхождане в ширина можем да построим покриващо дърво на G: на стъпка 1. инициализираме $D_0 = (\{r\}, \emptyset)$, след това ако сме построили $D_l = (V_l, E_l)$ на стъпка $l+1$ и $L_{l+1} = \{v_1, v_2, \ldots, v_p\}$, то за всеки връх $v_i \in L_{l+1}$ избираме по един съответен връх $u_i \in L_l$, такъв

че $\{u_i, v_i\} \in E$ (забележете, че тук за някои върхове v_i може да имаме повече от един възможен избор на v_i). Дървото на следващата стъпка е:

$$D_{l+1} = (V_l \cup L_{l+1}, E_l \cup \{\{u_1, v_1\}, \{u_2, v_2\}, \ldots, \{u_p, v_p\}\}).$$

Пример. Да разгледаме следния граф:

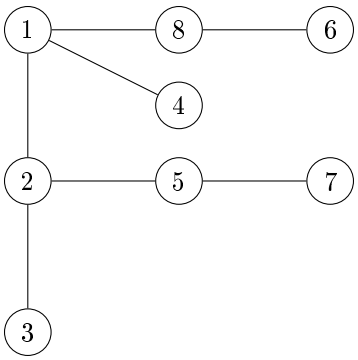


За да е ефективно обхождането, можем да представим графа със *списъци на съседите:*

$$1:2,4,8 \qquad 2:1,3,4,5 \qquad 3:2 \qquad 4:1,2,5,8$$
$$5:2,4,6,7 \qquad 6:5,7,8 \qquad 7:5,6 \qquad 8:1,4,6$$

Тъй като графът е неориентиран и няма структура на наследяване, всяко ребро участва в списъците и на двата си краища.

Избираме начален връх, например $r = 1$ и $L_0 = \{1\}$. Образуваме L_1, което се състои от всички необходени съседи на върховете от L_0, т.е $L_1 = \{2, 4, 8\}$. Следващото ниво L_2 се състои от необходените съседи на върховете от L_1. Един такъв връх е 3, който е съсед на 2. Друг такъв връх е 5 и забележете, че той е съсед както на върха 2, така и на върха 4. При построяването на покриващото дърво ще трябва да изберем едно от тези две ребра, например с бащата с по-малък номер, т.е. в случая да присъединим 5 към 2. Върхът 6 ще присъединим към 8 и така $L_2 = \{3, 5, 6\}$. Остава още една стъпка, при която $L_3 = \{7\}$ и той се присъединява към 5 (може и към 6). Построеното покриващо дърво изглежда така:



Реализацията на този алгоритъм се извършва с помощта на структурата от данни *опашка*. Опашката представлява списък, в който можем да изваждаме елементи само от началото и да вмъкваме елементи само в края. В началото в опашката е само началният връх 1. На всяка стъпка изваждаме елемента от началото на опашката и добавяме в нейния край всички необходени съседи на извадения връх. Така получаваме $2, 4, 8$, след това изваждаме 2 и добавяме необходените му съседи в края, получаваме $4, 8, 3, 5$ и така нататък. Този процес продължава докато в опашката има поне един елемент (обърнете внимание, че един връх влиза само един път в опашката, след което се счита за обходен).

Следващата ни цел ще бъде да докажем, че пътищата до всички върхове в покриващото дърво, построено в ширина, са оптимални по дължина (при фиксиран корен).

Дефиниция 3 *Нека $G(V, E)$ е неориентиран свързан граф. Дефинираме функцията $dist : V \times V \to \mathbb{N}$ по следния начин: за $u, v \in V$, $dist(u, v)$ е дължината на най-късия възможен път от u до v в графа G. Път от u до v с дължина $dist(u, v)$ се нарича* минимален.

Свойства на $dist$:

0) $dist(u, v)$ е добре дефинирано, тъй като графът G е свързан;

1) $dist(u, v) = dist(v, u)$, тъй като графът G е неориентиран;

2) $dist(u, v) = 0 \iff u = v$;

3) (*неравенство на триъгълника*) $dist(u, v) \leq dist(u, w) + dist(w, v)$;

Наистина, комбинирайки минимален път от u до w с минимален път от w до v получаваме път от u до v с дължина $dist(u, w) + dist(w, v)$, но $dist(u, v)$ е най-малката възможна дължина на такъв път. Забележете, че ако има минимален път от u до v през w, то имаме равенство, но ако такъв минимален път няма, то имаме строго неравенство.

4) (*принцип за оптималност*) Ако $dist(u, v) = k + 1$ и $u, \ldots, w, v$ е минимален път, то $dist(u, w) = k$.

С други думи, коя да е част от минимален път също трябва да е минимален път. В противен случай, можем да намалим дължината на целия път чрез намаляване на дължината на съответна част от него, която не е минимална.

Теорема 4 *Обхождането в ширина намира минимални пътища от началния връх r до всички върхове:*

$$\forall v \in V \; \forall i \leq l \; (dist(r, v) = i \iff v \in L_i).$$

Доказателство. Провеждаме три етапа.

Етап 1. $\forall i \leq l \; (v \in L_i \implies dist(r, v) \leq i)$.

Индукция по i. При $i = 0$ имаме $v \in L_0 \Rightarrow v = r \Rightarrow dist(r, v) = 0$. Нека твърдението е вярно за i. Да вземем произволен връх $v \in L_{i+1}$. Тогава

има връх $u \in L_i$, такъв че $\{u, v\} \in E$. От индукционното предположение, $dist(r, u) \leq i$, така че от свойство 3), $dist(r, v) \leq dist(r, u) + 1 \leq i + 1$.

Етап 2. $\forall i \leq l \; (dist(r, v) = i \implies v \in L_i)$.

Индукция по i. При $i = 0$ имаме $dist(r, v) = 0 \Rightarrow v = r \Rightarrow v \in L_0 = \{r\}$. Нека твърдението е вярно за i. Да вземем връх v, такъв че $dist(r, v) = i+1$. Има минимален път $r, \ldots, u, v$ от r до v. От свойство 4), $r, \ldots, u$ също е минимален път, така че $dist(r, u) = i$. От индукционното предположение, $u \in L_i$. Не е възможно $v \in L_j$ за $j \leq i$, тъй като от Етап 1 ще следва, че $dist(r, v) \leq j \leq i$, което противоречи на $dist(r, v) = i + 1$. По този начин, v е необходен връх при построяването на L_i, така че $v \in L_{i+1}$ (има ребро от $u \in L_i$ до v).

Етап 3. $\forall i \leq l \; (v \in L_i \implies dist(r, v) = i)$.

Нека $v \in L_i$. От Етап 1, $dist(r, v) = k \leq i$. От Етап 2 получаваме $v \in L_k$, но това е възможно само при $k = i$, тъй като $L_k \cap L_i = \emptyset$ при $k \neq i$.

Обхождане на граф в дълбочина

При *обхождането в дълбочина* основни понятия са: *текущ връх* (означаваме го с t) и *баща* $p(t)$ на текущия връх t. Считаме, че един връх $v \in V$ се обхожда в първия момент, в който v стане текущ, т.е. при първото присвояване $t = v$.

Процедурата е следната:

1. Избираме начален връх $r \in V$ и го обявяваме за текущ, т.е. $t = r$, като $p(t)$ е недефинирано.

2. Търсим необходен съсед $v \in V$ на текущия връх t.

2а. Ако има такъв v, неговият баща ще бъде t и v става текущ, т.е. $p(v) = t, \; t = v$. Отиваме отново към 2.

2б. Ако няма такъв v и $t \neq r$ извършваме *стъпка назад*: $t = p(t)$ и отиваме отново към 2.

2в. Ако няма такъв v и $t = r$, то обхождането приключва.

Ребрата $\{t, v\}$, които се използват на стъпка 2а по време на обхождането ще бъдат част от покриващото дърво D с корен r, построено в дълбочина. В началото $D = (\{r\}, \emptyset)$ и след това при всяко изпълняване на стъпка 2а присъединяваме v към t в дървото D. С други думи, в края на обхождането p ще бъде списък на бащите на D.

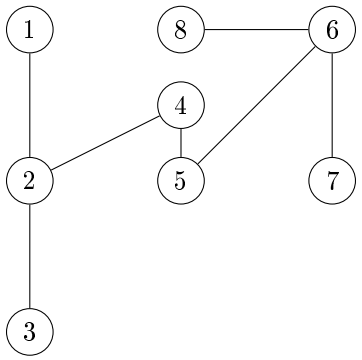
Разбира се, както отбелязохме при обхождането в ширина, покриващото дърво в общия случай не е единствено. На стъпка 2а е възможно да има повече от един необходени съседи, така че трябва да изберем някой от тях съобразно някакъв признак.

Пример. За горния граф с начален връх $r = 1$ по-долу са описани последователните стойности на текущия връх t. Показали сме ребрата, които

участват в покриващото дърво D, както и стъпките назад. Отново при повече от един необходим съсед избираме този, който има най-малък номер.

$t = 1$, $t = 2$ $\{1, 2\}$, $t = 3$ $\{2, 3\}$, $t = 2$ назад, $t = 4$ $\{2, 4\}$,
$t = 5$ $\{4, 5\}$, $t = 6$ $\{5, 6\}$, $t = 7$ $\{6, 7\}$, $t = 6$ назад, $t = 8$ $\{6, 8\}$,
$t = 6$ назад, $t = 5$ назад, $t = 4$ назад, $t = 2$ назад, $t = 1$ край.

Построеното покриващо дърво изглежда така:



За разлика от обхождането в ширина, тук дължините на пътищата от корена до останалите върхове в общия случай не са оптимални.

Реализацията на този алгоритъм се извършва с помощта на структурата от данни *стек*. Стекът е списък, при който изваждането и добавянето на елементи се случва само в единия край, който се нарича *връх* на стека. В началото стекът съдържа само коренът r. На всяка итерация на стъпка 2 текущият връх е във върха на стека. Ако има необходим съсед на текущия връх, той се добавя във върха на стека. Ако няма необходим съсед на текущия връх, т.е. при стъпка назад, той се изтрива от върха на стека. Краят на обхождането настъпва, когато стекът стане празен. За горния пример съдържанието на стека ще се изменя по следния начин:
1 ; $2, 1$; $3, 2, 1$; $2, 1$; $4, 2, 1$; $5, 4, 2, 1$; .....

Ойлерови обхождания

Дефиниция 5 *Нека $G = (V, E)$ е свързан неориентиран граф.*
Ойлеров път *в G наричаме всеки път, в който всяко ребро участва точно по веднъж. Ако началото и края на Ойлеровия път съвпадат, той се нарича* Ойлеров цикъл.
Казваме, че G е Ойлеров граф, *ако в G съществува Ойлеров цикъл.*

Теорема 6 (Ойлер) *Свързаният граф G е Ойлеров тогава и само тогава, когато степента $d(v)$ на всеки връх $v \in V$ е четна.*

Доказателство. ($\Longrightarrow$) Нека G е Ойлеров граф и да изберем Ойлеров цикъл в G. Да си представим участията на един връх $v \in V$ в този цикъл:

$$\ldots\ldots - v - \ldots\ldots - v - \ldots\ldots - v - \ldots\ldots$$

Всяко участие на v в цикъла определя две различни ребра с край v. Всяко ребро, което има край v по дефиниция трябва да участва точно веднъж в цикъла. По този начин, степента $d(v)$ е два пъти броя на участията на v в цикъла, което е четно число.

($\Longleftarrow$) Нека $d(v)$ е четно число за всеки връх $v \in V$. Ще опишем алгоритъм за построяване на Ойлеров цикъл, който е сходен с обхождането в дълбочина.

Започваме да изпълняваме следните стъпки:

1. Избираме произволен връх $r \in V$ и го обявяваме за текущ, $t = r$.

2. Докато е възможно: търсим съсед v на текущия връх t, за който реброто $\{t, v\}$ е необходено. Обхождаме реброто $\{t, v\}$ и обявяваме v за текущ, $t = v$.

Разбира се, стъпка 2 ще приключи в някакъв момент, тъй като ребрата са краен брой. Да допуснем, че стъпка 2 приключва в текущ връх $t \neq r$. Тогава построеният път изглежда така:

$$r - \ldots\ldots - t - \ldots\ldots - t - \ldots\ldots - t$$

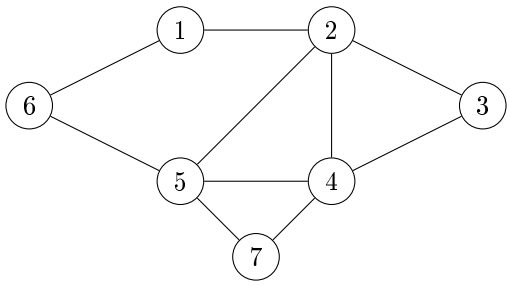
Всички ребра с край t трябва да са обходени точно по веднъж, от което излиза, че $d(t)$ е нечетно – противоречие.

И така в края на стъпка 2 имаме $t = r$, т.е. получили сме цикъл.

3. Ако този цикъл съдържа всички ребра, алгоритъмът приключва.

4. В противен случай, премахваме от G всички обходени ребра, участващи в цикъла. Полученият граф може да не е свързан, но отново всеки връх има четна степен. Търсим връх r', който участва в цикъла и от който излиза поне едно необходено ребро (такъв връх r' съществува, тъй като оригиналният граф е свързан). Обявяваме r' за текущ, $t = r'$ и отново изпълняваме стъпка 2. Получаваме нов цикъл с начало и край r', който вмъкваме на мястото на r' в стария цикъл. След това преминаваме към стъпка 3.

Пример. Да разгледаме следния граф:



Нека началният връх е $r = 1$. При повече от една възможности за необходени ребра, отново избираме онова ребро, чийто край има най-малък номер. На първата итерация получаваме цикъла: $1, 2, 3, 4, 5, 6, 1$. По-нататък трябва да изберем връх в този цикъл, от който излиза необходено ребро. Нека

това е $r' = 2$. На втората итерация получаваме цикъла: $2, 4, 7, 5, 2$. Заместваме в горния цикъл 2 с втория цикъл и получаваме: $1, 2, 4, 7, 5, 2, 3, 4, 5, 6, 1$. Тъй като няма повече необходени ребра, алгоритъмът приключва.